

Reliable date vs. number classification in tabular data

The state of the art for column type detection boils down to one principle: **never guess dates unless explicitly told to**. Every major library that has achieved zero false positives on numeric columns does so by either refusing to auto-detect dates entirely (pandas, PapaParse, DanfoJS), checking numbers *before* dates in a strict priority chain (D3, readr), or relying on preserved metadata rather than value inspection (SheetJS, readxl, openpyxl). The date-vs-number problem is fundamentally unsolvable from raw values alone — the number `44927` is indistinguishable from the date January 1, 2023 without format metadata. `Sheetjs` Tools that attempt aggressive date detection (older pandas `to_datetime`, Excel's CSV import, permissive `new Date()`) consistently produce false positives that corrupt data silently.

The libraries that got it right chose not to guess

The most reliable approach to date detection is, paradoxically, not to detect dates at all. **pandas** `read_csv()` **never attempts date inference during automatic type detection**. Its C-based parser follows a strict hierarchy — `int64 → float64 → bool → object` — and dates do not appear anywhere in that chain. `Artem Golubin` Date parsing only activates when the user explicitly passes `parse_dates=["column_name"]`. `DataScientYst` `Pandas` This is a deliberate design choice: pandas treats dates as an opt-in feature, not an automatic one, making numeric false positives impossible during standard CSV import.

PapaParse takes the same stance. Its `dynamicTyping` option converts numbers and booleans only `StudyRaid` `Papa Parse` — `+value` coercion for numbers, literal `"true"/"false"` matching for booleans. Date strings remain untouched as strings. DanfoJS follows suit, supporting only `float32`, `int32`, `string`, and `boolean` in its type system with no date type at all. `Jsdata`

D3's `autoType()` is the canonical example of a more aggressive but still safe approach. `Observable` Its complete implementation fits in roughly 15 lines of code, and the critical design decision is **checking numbers before dates**:

```
javascript

// Simplified D3 autoType logic
if (!value) value = null;
else if (value === "true") value = true;
else if (value === "false") value = false;
else if (!isNaN(number = +value)) value = number; // ← numbers FIRST
else if (value.match(ISO_8601_REGEX)) value = new Date(value);
```

Because `+value` converts any numeric string to a number (and `isNaN` confirms it), values like `"2024"` become the number `2024` rather than a Date. `GitHub` D3's date regex is deliberately narrow — it only matches ISO 8601 variants (`YYYY-MM-DD`, `YYYY-MM-DDTHH:MM:SS`) and requires hyphen separators. This means `"01/15/2024"` stays a string, `"2024"` becomes a number, and only `"2024-01-15"` becomes a Date. Arquero inherits this exact behavior since it delegates CSV parsing to D3.

R's readr applies the same numbers-before-dates principle. Its type-guessing hierarchy runs `logical → integer → double → number → date → datetime → time → character`, with doubles checked well before dates. [\(R for Data Science\)](#) Date detection requires ISO 8601 format with separators, so a column of integers like `20100101` stays as `double`, never becoming a date. [\(R-lib\)](#)

Spreadsheet files carry the answer — CSV files don't

For Excel/XLSX files, the date-vs-number distinction is a solved problem because **the metadata answers the question directly**. Excel has no separate date data type. Dates are IEEE 754 doubles where the integer part counts days since January 1, 1900, and the fractional part represents time as a fraction of 24 hours. The *only* mechanism that makes `44927` appear as "1/1/2023" is the **number format code** attached to the cell.

SheetJS uses its embedded SSF (SpreadSheet Format) library to examine this format metadata. The function `XLSX.SSF.is_date(format_string)` parses format strings character by character, looking for date/time tokens: `Y`, `M`, `D`, `H`, `S`, `AM/PM`, and calendar markers like `B1`/`B2`. When `cellDates: true` is set, any cell whose number format contains these tokens gets converted from type `'n'` (number) to type `'d'` (date), with the serial number replaced by a JavaScript Date object. [\(Sheetjs\)](#) Without `cellDates: true`, dates remain as plain numbers — a safe default that prevents false positives at the cost of requiring manual handling. [\(Sheetjs\)](#)

The `M` token in Excel format strings is **context-sensitive** — it means "month" by default but becomes "minutes" when adjacent to `H` (hours) or `S` (seconds). SheetJS resolves this by walking backwards through parsed tokens:

javascript

```
// SSF source: backward walk to resolve m (month) vs M (minute)
for(i = out.length-1, lst='t'; i >= 0; --i) {
  switch(out[i].t) {
    case 'h': case 'H': lst='h'; break;
    case 'm': if(lst === 's') out[i].t = 'M'; break; // minute if after seconds
  }
}
```

openpyxl uses the identical strategy in Python, checking `is_date_format(cell.number_format)` for tokens `d`, `y` (date) and `h`, `s` (time). [\(openpyxl\)](#) ExcelJS does the same via `cell.numFmt`. [\(StudyRaid\)](#) readxl skips inference entirely, reading cell types directly from Excel's format metadata and mapping datetime-formatted cells to R's `POSIXct` type. [\(GitHub\)](#)

This metadata-based approach is essentially bulletproof for binary Excel files. The problem emerges when exporting to CSV: format metadata is lost, and the formatted text `"1/1/2023"` or raw serial number `44927` carries no type information. [\(Clarusft\)](#) **This is why the SheetJS maintainer noted that `cellDates` was "originally designed for binary formats (XLS/XLSX/XLSB)" and only later extended to CSV.** [\(SheetJS\)](#) The fundamental lesson: use XLSX over CSV whenever possible, because the date question is already answered in the file. [\(Clarusft\)](#)

Why `new Date()` is the root cause of most false positives

JavaScript's `Date` constructor is notoriously permissive, and this permissiveness has cascaded into countless type-detection bugs:

javascript

```
new Date(42)    // → 42ms after Unix epoch (valid Date!)
new Date(0)     // → January 1, 1970 (valid Date!)
new Date(null)  // → January 1, 1970 (null → 0)
new Date(true)  // → 1ms after epoch (true → 1)
new Date("0")   // → varies by browser (!)
```

Any number passed to `new Date()` is treated as a millisecond timestamp. [GeeksforGeeks](#) This means **every integer in a dataset is a "valid" date** according to the constructor. Per the ECMAScript spec, only ISO 8601 format (`YYYY-MM-DDTHH:MM:SS`) is standardized for string parsing; all other formats are implementation-dependent, leading to cross-browser inconsistencies. [javaspring](#) MDN previously stated outright that "parsing of date strings with the Date constructor is strongly discouraged." [Matias Kinnunen](#)

The JavaScript date-parsing libraries have their own strictness problems. **Moment.js** without strict mode parses `"2016 is a date"` as a valid date when given format `"YYYY-MM-DD"`. [HTML Goodies](#) It even parses UUIDs as dates: `moment('5917238b-33ff-f849-cd63-80f4c9b37d0c', moment.ISO_8601)` produces `"5917-08-26..."`. [GitHub](#) Strict mode (`moment(value, format, true)`) fixes this by requiring exact format matches including separators, [GitHub](#) and the Moment.js team acknowledged that "more than half of parser issues on GitHub can be fixed by strict mode."

Day.js has a strict mode via the `customParseFormat` plugin, but it's unreliable — `dayjs('14970-44-22', 'YYYY-MM-DD', true).isValid()` returns `true` despite the impossible month 44. [GitHub](#) The strict mode internally compares `date !== this.format(format)`, which breaks with timezone offsets. [GitHub](#) **date-fns** lacks a built-in strict mode entirely; its `parseJSON('2020-02-30')` happily returns March 1 as valid. [GitHub](#) **chrono-node** takes the opposite approach as a natural language parser designed to extract dates from arbitrary text, [GitHub](#) making it far too permissive for type classification.

The practical implication: **never use `new Date()` or permissive date libraries as the sole mechanism for detecting whether a value is a date.** Always pre-filter with a structural check (regex or format matching) before attempting Date construction.

Design patterns that achieve zero false positives

The most reliable type-inference systems share several architectural patterns that, when combined, effectively eliminate false positives on numeric data.

Priority ordering with numbers before dates. This is the single most important pattern. D3, readr, and every zero-false-positive system checks `isNaN(+value)` or equivalent *before* attempting any date parsing. Since all date-like strings that are also valid numbers (like `"2024"` or `"42868"`) will match as numbers first, they never reach the date parser. The type hierarchy should be: `null/empty → boolean → number → date (strict format) → string`.

Conservative date regex over permissive constructors. D3's ISO 8601 regex — `/^([+]\d{2})?\d{4}(-\d{2}(-\d{2})?)?(T\d{2}:\d{2}(:\d{2}(\.\d{3})?)?(Z|[-+]\d{2}:\d{2})?)?$/` — is deliberately narrow. It requires separators, ruling out pure numeric strings. The recommended pattern for even stricter detection combines regex with round-trip validation: `jasvaspring`

```
javascript

function isStrictDate(str) {
  if (!/^\d{4}-\d{2}-\d{2}$/.test(str)) return false;
  const date = new Date(str + "T00:00:00Z");
  return date.toISOString().startsWith(str); // round-trip check
}
```

Sampling strategies matter significantly. Tools sample between 200 rows (Power BI) and all rows (Deephaven, Spark). readr's evolution is instructive: v1 sampled the first 1,000 rows sequentially and suffered frequent misclassifications because, as the readr authors noted, "through some application of Murphy's Law, many real CSV files have lots of `Readr` type changes after row 1000." Version 2 switched to **evenly-spaced sampling** throughout the file, `Readr` always including the last row — a dramatically better strategy. `Whu` Deephaven's two-pass approach (first pass infers types across all rows, second pass parses) represents the most thorough option: "you can't really be sure of the best-fitting data type for your column until you've looked at every last value." `Medium`

Tool	Sample size	Strategy
readr v2	1,000 rows	Evenly spaced through file
Tableau	10,000 (Excel) / 1,024 (CSV)	Sequential from top
Power BI	200 (CSV) / 1,000 (Excel)	Sequential from top
BigQuery	500 rows	Random file, first 500
Deephaven	All rows	Two-pass full scan
Spark	All rows	Full scan

Threshold-based confidence with safe defaults. The Frictionless Data `tableschema` library uses `confidence=0.85` — if 85% of sampled values parse as a type, the column gets that classification. `Hackers and Slackers` The `csv-schema-inference` library uses `acc=0.8`. `GitHub` But the zero-false-positive approach sets this threshold at **100%**: if any value fails parsing, the column falls through to the next type. pandas, readr, and D3 all require every value to match.

The "if in doubt, don't guess" principle. Hadley Wickham's readr documentation explicitly states: "I highly recommend always supplying `col_types`." `R for Data Science` Deephaven marks a column as String if multiple date formats appear. `Deephaven` Spark's `inferSchema` defaults to `false`, keeping everything as strings. `Palantir` This conservative philosophy treats type inference as a convenience hint, not a commitment.

Academic research points toward probabilistic and semantic approaches

The academic state of the art has moved beyond simple heuristics into machine learning territory. **Sherlock** (KDD 2019, MIT/IBM) trains a deep neural network on **686,765 data columns** from the VizNet corpus, extracting 1,588 features per column — statistical properties, character distributions, word embeddings, and paragraph vectors. It achieves a weighted F1 of **0.89** across 78 semantic types ([arXiv](#)) (not just syntactic types like "integer" but semantic types like "city" or "age"). This goes far beyond what regex-based approaches can achieve.

pptype (2020, Alan Turing Institute) introduces **Probabilistic Finite-State Machines** instead of regex patterns. ([The Alan Turing Institute](#)) Where regex produces binary accept/reject decisions, PFSMs assign probability scores to each value for each candidate type, enabling weighted posterior predictions. ([Springer](#)) This handles dirty data with missing values and anomalies gracefully — a column with one malformed date out of 10,000 won't collapse to "string." The follow-up **pptype-cat** (2021) extended this to categorical variable detection with improved date handling. ([University of Edinburgh](#))

Pythagoras (EDBT 2024) addresses a known weakness of prior methods: semantic type detection for purely numerical columns. Using Graph Neural Networks with a novel table graph representation, it leverages context — table names, neighboring column values — to predict whether a column of numbers represents prices, ages, dates, or quantities.

([Openproceedings](#)) This contextual approach mirrors the intuition that a column labeled "DOB" containing integers is more likely dates than quantities.

A large-scale study found that approximately **97% of columns** in real-world CSV files contain numeric, date, or character sequence data, ([The Alan Turing Institute](#)) confirming that the date-vs-number boundary is the most consequential type-inference decision for practical data processing. ([IOS Press](#))

A practical algorithm for zero-tolerance numeric safety

Synthesizing all the research, the state-of-the-art algorithm for column type classification with zero tolerance for numeric false positives follows this architecture:

Per-value type test (applied to every value in the column):

1. If empty/null/whitespace → [null](#) (skip for classification)
2. If literal ["true"](#)/["false"](#) → [boolean](#)
3. If [!isNaN\(+value\)](#) and value is not empty → [number](#) (**always before dates**)
4. If matches a strict date regex (ISO 8601 with required separators) AND passes round-trip validation → [date](#)
5. Otherwise → [string](#)

Per-column classification (aggregating per-value results):

1. Ignore null values for classification purposes
2. If **100%** of non-null values match a single type → classify as that type
3. If mixed types exist → classify as `string` (safe fallback)
4. If all values are null → classify as `string` with a warning

For Excel/XLSX files specifically: skip value-based inference entirely and read the cell's number format metadata. Use `SSF.is_date(format)` or equivalent to check for date tokens (`Y`, `M`, `D`, `H`, `S`). `Sheetjs` This is effectively 100% accurate because the file format preserves the author's intent.

Conclusion

The date-vs-number classification problem is not primarily a technical challenge — it's a **metadata problem**. When metadata exists (Excel number formats), the answer is unambiguous. When metadata is lost (CSV export), the problem becomes fundamentally underdetermined for certain value ranges. The tools that handle this best share three characteristics: they check numbers before dates in their priority chain, they use narrow regex patterns rather than permissive constructors, and they default to the least specific type (`string`) when uncertain. The academic frontier pushes toward probabilistic and ML-based approaches that can leverage column context and value distributions, but for production systems where false positives on numeric data are unacceptable, the simple rule remains: **if a value can be a number, it is a number — never a date.**